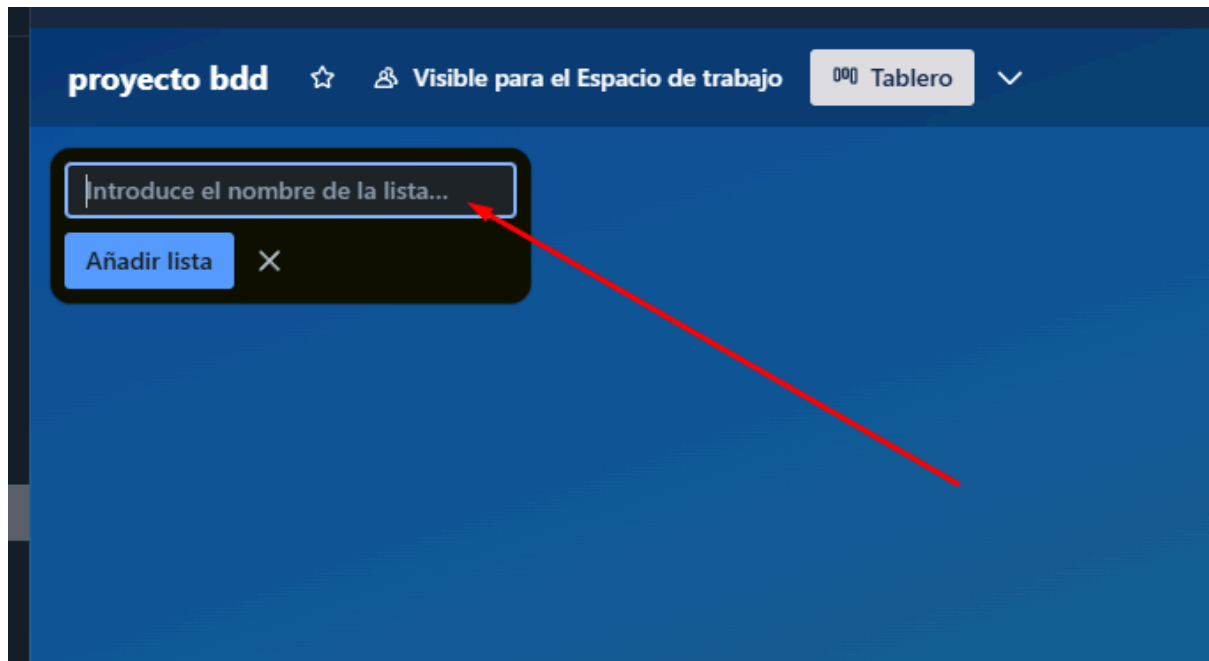


CON LA FECHA DE ENTREGA SE PUEDEN PASAR UNOS DIAS NO HAY PROBLEMA SOLO AVISEN

SI ESTAN EN PROCESO O DESARROLLO DE ALGO PONERLO EN EL TRELLO



Queries SQL Iniciales (SQL Server)

Primero, asegúrate de que las tablas se creen usando el script `db.sql` o `migration.sql` que proporcionaste.

Aquí tienes la query para insertar un usuario administrador con contraseña hasheada (usando el algoritmo SHA2_256 de SQL Server). Reemplaza `'contraseña_segura'` con la contraseña deseada antes de hashearla.

SQL

```
-- Asegúrate de estar usando la base de datos correcta
```

```
USE bdd_proyecto;
```

```
GO
```

```
-- Insertar el usuario administrador
```

```
-- Reemplaza 'admin@ejemplo.com' y 'contraseña_segura' con los valores deseados.
```

```
-- La contraseña se hashea usando SHA2_256. PHP necesitará comparar hashes usando el mismo algoritmo o preferiblemente usar password_hash() de PHP y almacenar ese resultado.
```

```
-- Si usas password_hash() de PHP, genera el hash en PHP y pégalo directamente aquí en lugar de HASHBYTES.
```

```
-- Ejemplo usando HASHBYTES (menos recomendado que password_hash de PHP para logins):
```

```
INSERT INTO [dbo].[Usuario] (
```

```
    [id_usuario],          -- Considera usar NEWID() si quieres IDs autogenerados tipo GUID
```

```

[nombre],
[apellido],
[email],
[password],
[rol],
[activo]          -- Asumiendo que quieres que esté activo por defecto
) VALUES (
  'admin001',      -- O usa NEWID() -> NEWID() si el tipo de dato fuera UNIQUEIDENTIFIER
  'Admin',
  'Principal',
  'admin@ejemplo.com',
  CONVERT(NVARCHAR(1000), HASHBYTES('SHA2_256', 'contraseña_segura'), 2), --
  Hashea 'contraseña_segura'
  'admin',
  1
);
GO

-- Query para verificar la inserción (opcional)
SELECT id_usuario, nombre, email, rol, activo FROM [dbo].[Usuario] WHERE email =
'admin@ejemplo.com';
GO

```

Nota sobre Hashing: Es altamente recomendable usar las funciones `password_hash()` y `password_verify()` de PHP para manejar las contraseñas. En ese caso, generarías el hash en PHP y lo insertarías como un string normal en la columna `password`. El `INSERT` de arriba es un ejemplo usando funciones SQL, pero la verificación en PHP sería más compleja si no usas `password_verify()`.

Plan de Trabajo Detallado (3 Semanas)

Semana 1: Fundamentos y Primeros CRUDs

1. Configuración Inicial y Login

Configurar el entorno de desarrollo, estructura del proyecto, conexión a la base de datos y crear el sistema de inicio de sesión con redirección por roles.

POSIBLE SOLUCION

1. Crear estructura de carpetas (p.ej., `config`, `controllers`, `views`, `public`, `models`).
2. Establecer la conexión a SQL Server en un archivo de configuración (`config/database.php`).
3. Crear la vista del formulario de login (`views/login.php`) con Tailwind CSS.
4. Crear un script PHP (`controllers/AuthController.php` o similar) que reciba los datos del formulario (POST).

5. Validar datos de entrada.
6. Consultar la tabla `Usuario` buscando el email.
7. Verificar la contraseña usando `password_verify()` (si se usó `password_hash()` de PHP) o comparando el hash (si se usó `HASHBYTES` de SQL).
8. Si las credenciales son válidas, iniciar una sesión (`session_start()`) y guardar datos del usuario (ID, nombre, rol).
9. Redirigir según el `rol` del usuario (`/admin/dashboard.php`, `/guia/home.php`, `/estudiante/home.php`).
10. Implementar un script para cerrar sesión.
11. Proteger las páginas que requieren autenticación verificando la existencia de la sesión.

PROMPT POSIBLE QUE FACILITE

"Genera el código PHP y HTML/Tailwind CSS para un sistema de inicio de sesión seguro. Debe:

1. Mostrar un formulario de login (email, password) estilizado con Tailwind.
2. Conectarse a una base de datos SQL Server (proporciona detalles de conexión genéricos).
3. Recibir los datos del formulario vía POST.
4. Validar que los campos no estén vacíos.
5. Buscar al usuario por email en la tabla 'Usuario'.
6. Verificar la contraseña usando `password_verify()` contra la columna 'password'.
7. Si es válido, iniciar sesión y redirigir según la columna 'rol' a `/admin/dashboard.php`, `/guia/home.php` o `/estudiante/home.php`.
8. Mostrar mensajes de error si el login falla.
9. Incluir un script para `logout.php` que destruya la sesión.
10. Muestra cómo proteger una página verificando si el usuario ha iniciado sesión."

FECHA ENTREGA

Día 1 - Día 2 (Miércoles 2 Abr - Jueves 3 Abr)

2. Dashboard Administrador

Crear la página principal del administrador que muestre estadísticas básicas y accesos directos.

POSIBLE SOLUCION

1. Crear la vista del dashboard (`views/admin/dashboard.php`) con layout básico y estilos Tailwind.
2. En el script PHP que carga esta vista (o al inicio de la vista), ejecutar queries SQL para obtener los conteos:

- `SELECT COUNT(*) as total_estudiantes FROM Usuario WHERE rol = 'estudiante';`
 - `SELECT COUNT(*) as total_eventos FROM Evento;`
 - `SELECT (SELECT COUNT(*) FROM ConvocatoriaGeneral) + (SELECT COUNT(*) FROM ConvocatoriaEspecifica) as total_convocatorias;`
3. Mostrar estos conteos en tarjetas o secciones de la página.
 4. Agregar botones o enlaces estilizados con Tailwind que apunten a las futuras secciones (Crear Evento, Reportes, etc.).

PROMPT POSIBLE QUE FACILITE

"Crea una página de dashboard de administrador usando PHP y Tailwind CSS. La página debe:

1. Conectarse a SQL Server.
2. Ejecutar las siguientes queries y mostrar los resultados:
 - `SELECT COUNT(*) as total_estudiantes FROM Usuario WHERE rol = 'estudiante';`
 - `SELECT COUNT(*) as total_eventos FROM Evento;`
 - `SELECT (SELECT COUNT(*) FROM ConvocatoriaGeneral) + (SELECT COUNT(*) FROM ConvocatoriaEspecifica) as total_convocatorias;`
3. Presentar cada conteo en una 'tarjeta' informativa estilizada con Tailwind.
4. Incluir botones/enlaces estilizados como 'Crear Nuevo Evento', 'Ver Reportes', 'Gestionar Usuarios' (aunque aún no funcionen)."

FECHA ENTREGA

Día 3 (Viernes 4 Abr)

3. Mantenimiento de Niveles Académicos (CRUD)

Crear la interfaz y lógica para Añadir, Ver, Editar y Eliminar Niveles Académicos.

POSIBLE SOLUCION

1. Crear un archivo PHP (`views/admin/niveles.php`).
2. **Mostrar Datos:** Al cargar la página, obtener todos los niveles de `NivelAcademico` y mostrarlos en una tabla HTML estilizada con Tailwind. Cada fila debe tener botones "Editar" y "Eliminar".
3. **Formulario (Crear/Editar):**
 - Incluir un formulario HTML (nombre del nivel).
 - Si se accede con un parámetro `edit_id` en la URL (`niveles.php?edit_id=xxx`):
 - Obtener los datos del nivel con ese ID.
 - Rellenar el formulario con esos datos.

- Cambiar el `action` del formulario para apuntar a una lógica de actualización (o usar un campo oculto `action_type='update'`).
 - Cambiar el texto del botón de envío a "Actualizar".
 - Si no hay `edit_id`, el formulario es para crear (acción por defecto, botón "Crear").
- 4. Lógica de Guardado/Actualización:**
- El formulario se envía (POST) a la misma página (`niveles.php`) o a un script controlador.
 - Verificar si es una acción de crear o actualizar (basado en el `action` o campo oculto).
 - Validar el nombre.
 - Ejecutar `INSERT` o `UPDATE` en la tabla `NivelAcademico`.
 - Redirigir de vuelta a `niveles.php` para mostrar la tabla actualizada (quizás con un mensaje de éxito).
- 5. Lógica de Eliminación:**
- El botón "Eliminar" de la tabla activa un modal de confirmación (JavaScript, p.ej., con SweetAlert2 o un modal simple de Tailwind).
 - Si se confirma, redirigir a un script (`niveles.php?delete_id=xxx` o un `delete_nivel.php?id=xxx`) que ejecute `DELETE FROM NivelAcademico WHERE id_nivelAcademico = xxx`.
 - Redirigir de vuelta a `niveles.php` con un mensaje.

PROMPT POSIBLE QUE FACILITE

"Genera el código PHP, HTML/Tailwind CSS y JavaScript para un CRUD completo de 'Nivel Académico' en una sola página (`niveles.php`). Debe:

1. Conectarse a SQL Server y mostrar los niveles existentes en una tabla Tailwind con botones Editar/Eliminar por fila.
2. Tener un formulario para crear/editar niveles (campo 'nombre').
3. Detectar un parámetro URL `edit_id`. Si existe, obtener los datos del nivel, rellenar el formulario para editar, y cambiar el botón a 'Actualizar'. Si no, el formulario es para crear.
4. Procesar el envío del formulario (POST) para insertar o actualizar en la tabla `NivelAcademico`.
5. Usar JavaScript para mostrar un modal de confirmación al hacer clic en 'Eliminar'. Si se confirma, realizar la eliminación (ejecutando un DELETE SQL) y refrescar la lista.
6. Redirigir a `niveles.php` después de cada operación con mensajes de éxito/error."

FECHA ENTREGA

Día 4 - Día 5 (Lunes 7 Abr - Martes 8 Abr)

Semana 2: CRUDs Dependientes y Eventos

4. Mantenimiento de Grados Académicos (CRUD)

Similar a Niveles, pero con dependencia de Nivel Académico.

POSIBLE SOLUCION

1. Sigue la misma estructura CRUD de Niveles (`views/admin/grados.php`).
2. **Formulario:** Añadir un campo `<select>` para `id_nivelAcademico`.
3. **Poblar Select:** Antes de mostrar el formulario (tanto para crear como para editar), obtener todos los `NivelAcademico` (`SELECT id_nivelAcademico, nombre FROM NivelAcademico`) y poblar las opciones del `<select>`.
4. **Guardar/Actualizar:** Incluir `id_nivelAcademico` en las sentencias `INSERT` y `UPDATE` para la tabla `Grado`.
5. **Mostrar Tabla:** Opcionalmente, hacer un `JOIN` en la query que obtiene los grados para mostrar también el nombre del nivel académico al que pertenecen (`SELECT G.*, N.nombre AS nombre_nivel FROM Grado G JOIN NivelAcademico N ON G.id_nivelAcademico = N.id_nivelAcademico`).

PROMPT POSIBLE QUE FACILITE

"Adapta el código CRUD anterior (de Niveles Académicos) para gestionar 'Grados Académicos' (`grados.php`). La tabla es 'Grado' (`id_grado`, `nombre`, `id_nivelAcademico`). Cambios necesarios:

1. El formulario debe incluir un campo `<select>` llamado `id_nivelAcademico`.
2. Poblar este `<select>` dinámicamente obteniendo los datos de la tabla 'NivelAcademico'.
3. Guardar/Actualizar el `id_nivelAcademico` seleccionado en la tabla 'Grado'.
4. Opcional: En la tabla que lista los grados, muestra también el nombre del 'Nivel Académico' asociado usando un `JOIN`."

FECHA ENTREGA

Día 6 (Miércoles 9 Abr)

5. Mantenimiento de Aulas (CRUD)

Similar a Grados, pero con dependencia de Grado.

POSIBLE SOLUCION

1. Sigue la misma estructura CRUD (`views/admin/aulas.php`). Tabla `Aula` (`id_aula`, `seccion`, `id_grado`).
2. **Formulario:** Añadir un campo `seccion` (texto) y un `<select>` para `id_grado`.

3. **Poblar Select:** Obtener todos los Grado (SELECT id_grado, nombre FROM Grado - quizás concatenar con el nivel para claridad) y poblar las opciones del <select>.
4. **Guardar/Actualizar:** Incluir seccion y id_grado en las sentencias INSERT y UPDATE para la tabla Aula.
5. **Mostrar Tabla:** Hacer JOIN con Grado (y potencialmente con NivelAcademico) para mostrar información descriptiva del grado/nivel al que pertenece el aula.

PROMPT POSIBLE QUE FACILITE

"Adapta el código CRUD anterior (de Grados) para gestionar 'Aulas' (aulas.php). La tabla es 'Aula' (id_aula, seccion, id_grado). Cambios necesarios:

1. El formulario debe tener un campo de texto 'seccion' y un <select> llamado id_grado.
2. Poblar el <select> de grados obteniendo los datos de la tabla 'Grado'. Considera mostrar el nombre del grado y su nivel para mayor claridad en las opciones.
3. Guardar/Actualizar la seccion y el id_grado seleccionado en la tabla 'Aula'.
4. En la tabla que lista las aulas, muestra la sección y el nombre del grado (y nivel) asociado usando JOINS."

FECHA ENTREGA

Día 7 (Jueves 10 Abr)

6. Mantenimiento de Tipos de Evento (CRUD)

CRUD simple para gestionar los tipos de evento y su tipo de asistencia.

POSIBLE SOLUCION

1. Sigue la estructura CRUD básica (views/admin/tipos_evento.php). Tabla TipoEvento (id_tipoEvento, nombre, tipoAsistencia).
2. **Formulario:** Campos para nombre (texto) y tipoAsistencia. tipoAsistencia puede ser un <select> con opciones fijas "QR" y "PADRE".
3. **Guardar/Actualizar:** Guardar ambos campos en la tabla TipoEvento.
4. **Mostrar Tabla:** Listar nombre y tipo de asistencia.

PROMPT POSIBLE QUE FACILITE

"Crea un CRUD PHP/Tailwind/SQL Server para 'Tipos de Evento' (tipos_evento.php). La tabla es 'TipoEvento' (id_tipoEvento, nombre, tipoAsistencia).

1. Sigue el patrón CRUD de una sola página con formulario y tabla.
2. El formulario debe tener un campo de texto 'nombre' y un <select> 'tipoAsistencia' con las opciones 'QR' y 'PADRE'.

3. Implementa la lógica para Crear, Leer (mostrar en tabla), Actualizar (con `edit_id`) y Eliminar (con modal de confirmación)."

FECHA ENTREGA

Día 8 (Viernes 11 Abr)

7. Mantenimiento de Eventos (CRUD)

Gestionar eventos, dependiendo de los Tipos de Evento. Mostrar tipo de asistencia en la tabla.

POSIBLE SOLUCION

1. Sigue la estructura CRUD (`views/admin/eventos.php`). Tabla `Evento` (`id_evento`, `nombre`, `descripcion`, `id_tipoEvento`).
2. **Formulario:** Campos `nombre` (texto), `descripcion` (textarea), y un `<select>` para `id_tipoEvento`.
3. **Poblar Select:** Obtener todos los `TipoEvento` (`SELECT id_tipoEvento, nombre FROM TipoEvento`) para poblar el `<select>`.
4. **Guardar/Actualizar:** Guardar los campos en la tabla `Evento`.
5. **Mostrar Tabla:** La query para obtener los eventos debe hacer un `JOIN` con `TipoEvento` para poder mostrar la columna `tipoAsistencia`.
6. SQL

```
SELECT E.*, TE.tipoAsistencia
FROM Evento E
INNER JOIN TipoEvento TE ON E.id_tipoEvento = TE.id_tipoEvento;
```

- 7.
8. Mostrar `E.nombre`, `E.descripcion`, y `TE.tipoAsistencia` en las columnas de la tabla.

PROMPT POSIBLE QUE FACILITE

"Adapta el código CRUD para gestionar 'Eventos' (`eventos.php`). La tabla es 'Evento' (`id_evento`, `nombre`, `descripcion`, `id_tipoEvento`).

1. El formulario debe incluir campos para nombre, descripción y un `<select>` 'id_tipoEvento' poblado desde la tabla 'TipoEvento'.
2. Implementa la lógica CRUD completa (Crear, Leer, Actualizar, Eliminar).
3. En la tabla que lista los eventos, en lugar de mostrar el ID del tipo de evento, muestra directamente el campo 'tipoAsistencia' obtenido de la tabla 'TipoEvento' usando un `INNER JOIN` en la consulta `SELECT`."

FECHA ENTREGA

Semana 3: Convocatorias, Asignaciones y Formularios Complejos

8. Mantenimiento de Convocatoria General (CRUD)

Gestionar convocatorias generales, dependiendo de Nivel Académico y Evento.
Mostrar tipo de asistencia.

POSIBLE SOLUCION

1. Estructura CRUD (views/admin/convocatorias_generales.php). Tabla ConvocatoriaGeneral (id_convocatoriaGeneral, lugar, fecha, id_evento, id_nivelAcademico).
2. **Formulario:** Campos lugar (texto), fecha (usar un input datetime-local), <select> para id_evento, <select> para id_nivelAcademico.
3. **Poblar Selects:** Obtener datos de Evento y NivelAcademico para poblar los respectivos <select>.
4. **Guardar/Actualizar:** Guardar todos los campos. Asegurarse de que el formato de fecha sea compatible con SQL Server DATETIME2.
5. **Mostrar Tabla:** La query necesita JOIN con Evento y luego con TipoEvento para obtener tipoAsistencia. También puede hacer JOIN con NivelAcademico para mostrar el nombre del nivel.
6. SQL

```
SELECT CG.*, NA.nombre AS nombre_nivel, E.nombre AS nombre_evento, TE.tipoAsistencia
FROM ConvocatoriaGeneral CG
INNER JOIN NivelAcademico NA ON CG.id_nivelAcademico = NA.id_nivelAcademico
INNER JOIN Evento E ON CG.id_evento = E.id_evento
INNER JOIN TipoEvento TE ON E.id_tipoEvento = TE.id_tipoEvento;
```

- 7.
8. Mostrar columnas relevantes como lugar, fecha, nombre_nivel, nombre_evento, tipoAsistencia.

PROMPT POSIBLE QUE FACILITE

"Crea un CRUD PHP/Tailwind/SQL Server para 'Convocatoria General' (convocatorias_generales.php). Tabla: 'ConvocatoriaGeneral' (id_convocatoriaGeneral, lugar, fecha, id_evento, id_nivelAcademico).

1. Sigue el patrón CRUD de una sola página.
2. El formulario debe incluir: campo de texto 'lugar', un input 'datetime-local' para 'fecha', un <select> 'id_evento' (poblado desde 'Evento'), y un <select> 'id_nivelAcademico' (poblado desde 'NivelAcademico').

3. Implementa la lógica CRUD completa. Maneja correctamente el formato de fecha/hora para SQL Server.
4. En la tabla de resultados, muestra lugar, fecha, nombre del nivel, nombre del evento y el 'tipoAsistencia' del evento asociado, usando los JOINS necesarios."

FECHA ENTREGA

Día 11 (Miércoles 16 Abr)

9. Mantenimiento de Convocatoria Específica (CRUD)

Similar a la General, pero depende de Aula y Evento. Mostrar tipo de asistencia.

POSIBLE SOLUCION

1. Estructura CRUD (views/admin/convocatorias_especificas.php). Tabla `ConvocatoriaEspecifica` (id_convocatoriaEspecifica, lugar, fecha, id_evento, id_aula).
2. **Formulario:** Campos `lugar`, `fecha` (datetime-local), `<select>` para `id_evento`, `<select>` para `id_aula`.
3. **Poblar Selects:** Obtener datos de `Evento` y `Aula` (quizás mostrando sección, grado y nivel para identificarla bien).
4. **Guardar/Actualizar:** Guardar todos los campos.
5. **Mostrar Tabla:** Query con `JOIN` a `Aula`, `Evento`, y `TipoEvento`. Puede necesitar `JOIN` adicional a `Grado` y `NivelAcademico` si se quiere mostrar la descripción completa del aula.
6. SQL

```
SELECT CE.*, A.seccion, G.nombre AS nombre_grado, E.nombre AS nombre_evento,
TE.tipoAsistencia
FROM ConvocatoriaEspecifica CE
INNER JOIN Aula A ON CE.id_aula = A.id_aula
INNER JOIN Grado G ON A.id_grado = G.id_grado -- Opcional, para más detalle
INNER JOIN Evento E ON CE.id_evento = E.id_evento
INNER JOIN TipoEvento TE ON E.id_tipoEvento = TE.id_tipoEvento;
```

7.

8. Mostrar columnas relevantes.

PROMPT POSIBLE QUE FACILITE

"Crea un CRUD PHP/Tailwind/SQL Server para 'Convocatoria Específica' (convocatorias_especificas.php). Tabla: 'ConvocatoriaEspecifica' (id_convocatoriaEspecifica, lugar, fecha, id_evento, id_aula).

1. Sigue el patrón CRUD de una sola página.

2. El formulario debe incluir: 'lugar', 'fecha' (datetime-local), `<select>` 'id_evento', y `<select>` 'id_aula' (poblado desde 'Aula', mostrando detalles identificativos como sección/grado).
3. Implementa la lógica CRUD completa.
4. En la tabla de resultados, muestra lugar, fecha, detalles del aula (sección/grado), nombre del evento y el 'tipoAsistencia' del evento asociado, usando los JOINS necesarios."

FECHA ENTREGA

Día 12 (Jueves 17 Abr)

10. Asignación Guía a Aula

Interfaz para seleccionar un usuario (rol='guia') y un aula y crear la relación en `UsuarioAula`.

POSIBLE SOLUCION

1. Crear página (`views/admin/asignar_guia_aula.php`).
2. **Formulario:** Dos campos `<select>` o inputs con autocompletado: uno para seleccionar Usuario (filtrado por `rol='guia'`) y otro para seleccionar Aula.
3. **Poblar Selects/Autocompletado:**
 - Obtener Usuarios: `SELECT id_usuario, nombre, apellido FROM Usuario WHERE rol = 'guia' AND activo = 1;`
 - Obtener Aulas: `SELECT A.id_aula, A.seccion, G.nombre AS nombre_grado FROM Aula A JOIN Grado G ON A.id_grado = G.id_grado;` (Mostrar info descriptiva).
 - Para autocompletado: Crear endpoints PHP (`api/search_guias.php`, `api/search_aulas.php`) que reciban un término de búsqueda y devuelvan JSON. Usar JavaScript (Fetch API + alguna librería como `TomSelect/Select2` o `simple datalist`) para llamar a estos endpoints y mostrar sugerencias.
4. **Guardar:** Al enviar el formulario, obtener `id_usuario` y `id_aula` seleccionados e insertar en `UsuarioAula` (asegúrate que la PK `id_usuarioaula` sea única, quizás usando `NEWID()` si es `UNIQUEIDENTIFIER` o un autoincremental). Prevenir duplicados.
5. **Mostrar Asignaciones:** Debajo del formulario, mostrar una tabla con las asignaciones existentes (`SELECT UA.id_usuarioaula, U.nombre, U.apellido, A.seccion, G.nombre AS nombre_grado FROM UsuarioAula UA JOIN Usuario U ON UA.id_usuario = U.id_usuario JOIN Aula A ON UA.id_aula = A.id_aula JOIN Grado G ON A.id_grado = G.id_grado`). Añadir botón para eliminar asignación.

PROMPT POSIBLE QUE FACILITE

"Crea una página PHP/Tailwind para asignar Usuarios (con rol 'guia') a Aulas (asignar_guia_aula.php). Debe:

1. Mostrar un formulario con dos campos para seleccionar un Guía y un Aula. Implementa autocompletado para ambos campos usando JavaScript y endpoints PHP que devuelvan JSON (uno para buscar guías por nombre/apellido, otro para buscar aulas por sección/grado).
2. Al enviar, insertar la relación en la tabla 'UsuarioAula', previniendo duplicados.
3. Mostrar una tabla con las asignaciones existentes (Nombre Guía, Apellido Guía, Sección Aula, Grado Aula) con un botón para eliminar cada asignación.
4. Implementar la lógica de eliminación para registros en 'UsuarioAula'."

FECHA ENTREGA

Día 13 (Viernes 18 Abr)

11. Formulario Recolección Datos Padres/Hijos

Formulario complejo para registrar un padre y múltiples hijos (estudiantes), relacionándolos.

POSIBLE SOLUCION

1. Crear la página del formulario (views/public/registro_familia.php o similar). Usar mucho JavaScript para la dinámica.
2. **Sección Padre:** Formulario con campos para PadreFamilia (dui, nombre, apellidos).
3. **Sección Hijos (Dinámica):**
 - Un botón "Añadir Hijo".
 - Al hacer clic, JavaScript clona/crea un bloque de formulario para un hijo. Campos: nombre, apellido, email, Nivel Académico (<select>), Grado (<select>), Aula (<select>).
 - Implementar **cascading selects**:
 - Al seleccionar Nivel, usar Fetch API para llamar a un endpoint PHP (api/get_grados.php?nivel_id=X) que devuelve los grados de ese nivel. Poblar el select de Grado.
 - Al seleccionar Grado, usar Fetch API para llamar a (api/get_aulas.php?grado_id=Y) que devuelve las aulas de ese grado. Poblar el select de Aula.
 - Un botón "Eliminar Hijo" en cada bloque de hijo.
4. **Envío:** JavaScript recopila todos los datos (padre + array de hijos con sus datos y aula seleccionada) y los envía (probablemente como JSON) a un script PHP (controllers/registroFamiliaController.php).
5. **Procesamiento Backend (PHP):**
 - Iniciar una transacción SQL.

- Validar todos los datos recibidos.
- Insertar en `PadreFamilia`. Obtener el `id_padreFamilia` generado.
- Iterar sobre los datos de los hijos:
 - Generar/hashear una contraseña inicial o dejarla pendiente.
 - Insertar en `Usuario` (con `rol='estudiante'`, `activo=1`). Obtener el `id_usuario` generado.
 - Insertar en `UsuarioAula` la relación del estudiante con el `id_aula` seleccionado en el formulario.
 - Insertar en `UsuarioPadresFamilia` la relación entre el `id_usuario` del hijo y el `id_padreFamilia`.
- Si todo va bien, `COMMIT` la transacción. Si hay error, `ROLLBACK`.
- Redirigir a una página de éxito o mostrar mensaje.

PROMPT POSIBLE QUE FACILITE

"Desarrolla un formulario complejo de registro familiar (`registro_familia.php`) usando HTML, Tailwind CSS y JavaScript intensivo. Debe:

1. Tener campos para los datos de 'PadreFamilia' (dui, nombre, apellidos).
2. Permitir añadir dinámicamente múltiples formularios para 'Hijos' (nombre, apellido, email, selects para Nivel, Grado, Aula).
3. Implementar selects en cascada para Nivel->Grado->Aula usando Fetch API y endpoints PHP (`api/get_grados.php`, `api/get_aulas.php`).
4. Permitir eliminar hijos añadidos dinámicamente.
5. Al enviar, recopilar todos los datos (padre y array de hijos) con JavaScript y enviarlos a un script PHP (`registro_familia_action.php`).
6. El script PHP debe:
 - Usar transacciones SQL Server.
 - Validar datos.
 - Insertar el 'PadreFamilia'.
 - Iterar por los hijos: insertar en 'Usuario' (`rol='estudiante'`), insertar en 'UsuarioAula', insertar en 'UsuarioPadresFamilia' relacionando hijo y padre.
 - Realizar `COMMIT` o `ROLLBACK`.
 - Mostrar mensaje de éxito/error."

FECHA ENTREGA

Día 14 - Día 15+ (Lunes 21 Abr - Martes 22 Abr en adelante - Esta tarea es la más compleja y podría extenderse o requerir más esfuerzo).

Consideraciones Finales:

- **Buffer:** No hay mucho tiempo de margen en este plan. Cualquier retraso en una tarea afectará a las siguientes.
- **Testing:** Se debe dedicar tiempo a probar cada funcionalidad.
- **Estilos y UX:** Tailwind ayuda, pero refinar la experiencia de usuario puede llevar tiempo adicional.
- **Seguridad:** Asegúrate de validar todas las entradas del usuario (cliente y servidor), prevenir inyección SQL (usando consultas preparadas), proteger contra XSS y CSRF.
- **IDs:** El script SQL usa `NVARCHAR(1000)` para las claves primarias. Considera si `INT IDENTITY` o `UNIQUEIDENTIFIER` (con `NEWID()`) serían más apropiados para algunas tablas, especialmente para IDs autogenerados.

Este plan te da una estructura detallada. ¡Mucho éxito con el proyecto!